

LAB ACTIVITY (Exp-04)

1. Given a string s containing three types of brackets {}, () and []. Determine whether the Expression are balanced not. An expression is balanced if each opening bracket has a corresponding closing bracket of the same type, the pairs are properly ordered and no bracket closes before its matching opening bracket.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <string.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char c) {
    if (top == MAX - 1) {
        printf("Stack overflow\n");
        exit(1);
    }
    stack[++top] = c;
}

char pop() {
    if (top == -1) {
        printf("Stack underflow\n");
        exit(1);
    }
    return stack[top--];
}

bool isEmpty() {
    return top == -1;
}

bool isBalanced(char *exp) {
    for (int i = 0; exp[i] != '\0'; i++) {
        char ch = exp[i];

        if (ch == '(' || ch == '{' || ch == '[') {
            push(ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']') {
            if (isEmpty()) return false;

            char topChar = pop();

            if ((ch == ')' && topChar != '(') ||
                (ch == '}' && topChar != '{') ||
                (ch == ']' && topChar != '['))
                return false;
        }
    }

    return isEmpty();
}
```

```

        (ch == ']' && topChar != '[')) {
            return false;
        }
    }
}

return isEmpty();
}

int main() {
    char exp[MAX];

    printf("Enter an expression: ");
    scanf("%s", exp);

    if (isBalanced(exp)) {
        printf("Balanced\n");
    } else {
        printf("Not Balanced\n");
    }

    return 0;
}

```

```

PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)> cd "Data structures and algorithm\lab5(2409)" ; if ($?) { gcc balanced.c -o balanced.exe }
Enter an expression: [{()}]
Balanced
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)> cd "Data structures and algorithm\lab5(2409)" ; if ($?) { gcc balanced.c -o balanced.exe }
Enter an expression: ([{}])
Not Balanced
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)>

```

2. Given a stack with push(), pop(), and empty() operations, The task is to delete the middle element of it without using any additional data structure.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int stack[MAX];
int top = -1;

void push(int x) {
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = x;
}

int pop() {
    if (top == -1) {
        printf("Stack Underflow\n");
        return -1;
    }
    return stack[top--];
}

int isEmpty() {
    return top == -1;
}

void deleteMiddleUntil(int current, int size) {
    if (isEmpty() || current == size) {
        return;
    }

    int x = pop();

    if (current != size / 2) {
        deleteMiddleUntil(current + 1, size);
        push(x);
    } else {
        deleteMiddleUntil(current + 1, size);
    }
}

void deleteMiddle() {
    int size = top + 1;
    deleteMiddleUntil(0, size);
}

void printStack() {
    for (int i = top; i >= 0; i--) {
        printf("%d ", stack[i]);
    }
    printf("\n");
}
```

```

}

int main() {
    int n, val;

    printf("Enter number of elements in stack: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        push(val);
    }

    printf("\nOriginal Stack (Top to Bottom): ");
    printStack();

    deleteMiddle();

    printf("After Deleting Middle Element: ");
    printStack();

    return 0;
}

```

```

PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)> cd "d:
tructures and algorithm\lab5(2409)\\" ; if ($?) { gcc delmiddle.c -o delmid
le }
Enter number of elements in stack: 5
Enter 5 elements: 10 20 30 40 50

Original Stack (Top to Bottom): 50 40 30 20 10
After Deleting Middle Element: 50 40 20 10
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)> cd "d:
tructures and algorithm\lab5(2409)\\" ; if ($?) { gcc delmiddle.c -o delmid
le }
Enter number of elements in stack: 10
Enter 10 elements: 5 9 6 7 6 6 5 10 12 9

Original Stack (Top to Bottom): 9 12 10 5 6 6 7 6 9 5
After Deleting Middle Element: 9 12 10 5 6 7 6 9 5
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)>

```

3. Implement Stack operations using Linked List.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* top = NULL;

void push(int x) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = x;
    newNode->next = top;
    top = newNode;
}

int pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return -1;
    }
    struct Node* temp = top;
    int popped = temp->data;
    top = top->next;
    free(temp);
    return popped;
}

int peek() {
    if (top == NULL) {
        printf("Stack is empty\n");
        return -1;
    }
    return top->data;
}

void display() {
    struct Node* temp = top;
    if (temp == NULL) {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack (Top to Bottom): [");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("]\n");
}

int main() {
    int n, val;
```

```

printf("Enter number of elements in stack: ");
scanf("%d", &n);

printf("Enter %d elements:\n", n);
for (int i = 0; i < n; i++) {
    scanf("%d", &val);
    push(val);
}

printf("\nStack created successfully!\n");
display();

printf("Top element is %d\n", peek());

printf("Popped element is %d\n", pop());
display();

return 0;
}

```

```

PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)> cd
tructures and algorithm\lab5(2409)\\" ; if ($?) { gcc stacklinkedlist.c
{ .\stacklinkedlist }
Enter number of elements in stack: 5
Enter 5 elements:
10 20 30 40 50

Stack created successfully!
Stack (Top to Bottom): [50 40 30 20 10 ]
Top element is 50
Popped element is 50
Stack (Top to Bottom): [40 30 20 10 ]
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)>

```